

DOCUMENTAÇÃO TÉCNICA DA API

Edook

Sumário

1	Introdução	3
2	Visão Geral do Backend	3
2.1	Objetivo da API	3
2.2	Escopo Funcional	3
3	Arquitetura do Sistema	3
3.1	Arquitetura Lógica	3
3.2	Descrição das Camadas	4
4	Tecnologias Utilizadas	4
5	Estrutura do Código	5
6	Autenticação e Segurança	6
6.1	Modelo de Autenticação	6
6.2	Boas Práticas de Segurança	6
7	Referências da API	6
7.1	Visão Geral dos Recursos	6
7.2	Resumo dos Endpoints	6
8	Especificação Detalhada dos Endpoints	7
8.1	Auth	7
8.1.1	Efetuar Login	7
8.2	Funcionários	8
8.2.1	Cadastrar Funcionário	8
8.2.2	Buscar Funcionário	8
8.2.3	Atualizar Funcionário	8
8.2.4	Excluir Funcionário	9
8.2.5	Enviar Código	9
8.2.6	Verificar Código	9
8.2.7	Reenviar Confirmação	10
8.2.8	Confirmar E-mail	10
8.2.9	Redefinir Senha	10

8.3	Equipamentos	11
8.3.1	Listar Equipamentos	11
8.3.2	Cadastrar Equipamento	11
8.3.3	Atualizar Equipamento	11
8.3.4	Excluir Equipamento	12
8.4	Reservas	12
8.4.1	Listar Reservas	12
8.4.2	Criar Reserva	12
8.4.3	Cancelar Reserva	12
9	Execução do Backend	13
9.1	Pré-requisitos	13
9.2	Configuração do Envio de E-mails	13
9.3	Inicialização da API	13
9.4	Acesso à Documentação Swagger	14
9.5	Acesso ao Banco de Dados	14
9.6	Encerramento da Aplicação	14
10	Validações dos DTOs	14

1 Introdução

Este documento descreve a estrutura técnica do backend da API do aplicativo Edook. O objetivo é registrar a arquitetura geral, as tecnologias utilizadas, a organização do código e a especificação dos endpoints disponíveis para autenticação, gerenciamento de funcionários, gerenciamento de equipamentos e controle de reservas.

A documentação pode ser vista através de uma interface amigável gerada pelo Swagger da aplicação, cuja URL base em ambiente local é <http://localhost:8080/swagger-ui/index.html>. **Essa interface só é possível de ser visualizada com a API em execução.**

2 Visão Geral do Backend

2.1 Objetivo da API

A API do Edook é responsável por centralizar as regras de negócio do sistema, processar requisições HTTP, validar dados de entrada, autenticar usuários, persistir informações e retornar respostas em formato JSON. O backend disponibiliza recursos para login, confirmação de e-mail, recuperação de senha, cadastro de funcionários, cadastro de equipamentos e criação/cancelamento de reservas.

2.2 Escopo Funcional

- Autenticação de usuários por identificador e senha;
- Cadastro, busca, atualização e exclusão de funcionários;
- Envio, verificação, reenvio e confirmação de código por e-mail;
- Redefinição de senha de funcionários;
- Cadastro, listagem, atualização e exclusão de equipamentos;
- Criação, listagem e cancelamento de reservas de equipamentos.

3 Arquitetura do Sistema

3.1 Arquitetura Lógica

O backend segue uma organização em camadas. Essa separação facilita a manutenção, os testes, a evolução do sistema e a divisão clara de responsabilidades

entre entrada HTTP, regras de aplicação, domínio e persistência.

3.2 Descrição das Camadas

A estrutura do backend do Edook está organizada no pacote principal `com.pi1.Edook`. Dentro dele, as responsabilidades são separadas em pacotes específicos, conforme a arquitetura apresentada no projeto.

- **config:** concentra classes de configuração da aplicação como beans e OpenApi.
- **controller:** representa a camada de apresentação da API. Contém os controllers REST responsáveis por receber requisições HTTP, acionar os serviços e retornar respostas ao cliente.
- **dto:** reúne os objetos de transferência de dados usados nas entradas e saídas da API, como DTOs de criação, atualização, login, resposta e verificação de código.
- **exception:** centraliza exceções e tratamentos de erro da aplicação, permitindo padronizar respostas em situações de falha ou violação de regras de negócio.
- **model:** contém as entidades e modelos principais do domínio, como funcionário, equipamento e reserva.
- **repository:** implementa a camada de acesso a dados, responsável pela comunicação com o banco de dados e pelas operações de persistência.
- **service:** concentra as regras de negócio e os casos de uso da aplicação, fazendo a ponte entre controllers, repositories, models e DTOs.
- **EdookApplication.java:** classe principal responsável por inicializar a aplicação Spring Boot.

4 Tecnologias Utilizadas

Categoria	Tecnologia / Descrição
Linguagem	Java 21.
Framework Backend	Spring Boot, Spring Data JPA e controllers REST.
Formato de Comunicação	JSON sobre HTTP.
Documentação	OpenAPI 3.1.0, gerada pelo Swagger.

Servidor Local	http://localhost:8080.
Banco de Dados	PostgreSQL.
Containerização	Docker e Docker Compose.
Autenticação	Login com retorno de token para uso nas requisições protegidas.
Validação	Validações declarativas nos DTOs, incluindo campos obrigatórios, e-mail, padrões de telefone e senha.

5 Estrutura do Código

A estrutura do backend do Edook segue a organização abaixo, conforme a disposição dos diretórios do projeto. Além do código-fonte Java, o projeto possui scripts SQL de inicialização do banco de dados, documentação e arquivos de configuração da aplicação.

```

banco_de_dados/
|-- init.sql

docs/

src/
'-- main/
    |-- java/com/pi1/Edook/
    |   |-- config/
    |   |-- controller/
    |   |-- dto/
    |   |-- exception/
    |   |-- model/
    |   |-- repository/
    |   |-- service/
    |   '-- EdookApplication.java
'-- resources/
    '-- application.properties

```

- **banco_de_dados/init.sql:** script utilizado para criação das tabelas e inserções no banco de dados.
- **docs:** diretório reservado para documentação do projeto.
- **src/main/java/com/pi1/Edook:** pacote principal do backend, onde ficam as camadas da aplicação.

- **src/main/resources/application.properties:** arquivo de configuração da aplicação Spring Boot.

6 Autenticação e Segurança

6.1 Modelo de Autenticação

A autenticação é realizada pelo endpoint `POST /auth/login`. O usuário informa um identificador e uma senha. Em caso de sucesso, a API retorna os dados básicos do funcionário autenticado e um token de acesso.

6.2 Boas Práticas de Segurança

- Senhas devem ser armazenadas com hash seguro;
- Tokens devem possuir tempo de expiração;
- Dados sensíveis não devem ser retornados em respostas públicas;
- Operações de atualização e exclusão devem validar permissões;
- Códigos de verificação devem possuir validade limitada.

7 Referências da API

7.1 Visão Geral dos Recursos

Recurso	Responsabilidade
Auth	Autenticação de usuários e geração de token.
Funcionários	Cadastro, busca, atualização, exclusão, confirmação de e-mail e recuperação de senha.
Equipamentos	Cadastro, listagem, atualização e exclusão de equipamentos.
Reservas	Criação, listagem e cancelamento de reservas de equipamentos.

7.2 Resumo dos Endpoints

Método	Rota	Descrição
POST	/auth/login	Autentica um usuário.
POST	/funcionarios	Cadastra um funcionário.
GET	/funcionarios/busca	Busca funcionário por identificador.
PUT	/funcionarios/{cpf}	Atualiza dados de um funcionário.
DELETE	/funcionarios/{cpf}	Exclui um funcionário.
POST	/funcionarios/enviar-codigo	Envia código de verificação.
POST	/funcionarios/verificar-codigo	Verifica código enviado.
POST	/funcionarios/reenviar-confirmacao	Reenvia código de confirmação.
PUT	/funcionarios/confirmar-email	Confirma e-mail do funcionário.
PATCH	/funcionarios/redefinir-senha	Redefine senha.
GET	/equipamentos	Lista equipamentos.
POST	/equipamentos	Cadastra equipamento.
PUT	/equipamentos/{prefixo}/{numero}	Atualiza equipamento.
DELETE	/equipamentos/{prefixo}/{numero}	Exclui equipamento.
GET	/reservas	Lista reservas.
POST	/reservas	Cria reserva.
PATCH	/reservas/{id}/cancelar	Cancela reserva.

8 Especificação Detalhada dos Endpoints

8.1 Auth

8.1.1 Efetuar Login

Método	POST
Rota	/auth/login

Finalidade: autentica um funcionário no sistema por identificador e senha. Em caso de sucesso, retorna os dados do funcionário e um token.

Corpo da Requisição:

```
{
```

```
"identificador": "string",  
"senha": "string"  
}
```

8.2 Funcionários

8.2.1 Cadastrar Funcionário

Método POST
Rota /funcionarios

Finalidade: cadastra um novo funcionário no sistema. O cadastro exige dados pessoais, contato, cargo, matrícula, senha e código de verificação.

Corpo da Requisição:

```
{  
  "nome": "string",  
  "cpf": "string",  
  "email": "usuario@email.com",  
  "senha": "Senha@123",  
  "ddd": "85",  
  "numero": "999999999",  
  "cargo": "string",  
  "matricula": 123,  
  "codigoVerificacao": "string"  
}
```

8.2.2 Buscar Funcionário

Método GET
Rota /funcionarios/busca?identificador={valor}

Finalidade: busca um funcionário a partir de um identificador informado por parâmetro de consulta.

Parâmetros:

Nome	Local	Descrição
identificador	Query	Valor usado para localizar o funcionário.

8.2.3 Atualizar Funcionário

Método PUT
Rota /funcionarios/{cpf}

Finalidade: atualiza nome e telefone de um funcionário identificado pelo CPF.

Parâmetros:

Nome	Local	Descrição
cpf	Path	CPF do funcionário que será atualizado.

Corpo da Requisição:

```
{
  "nome": "string",
  "ddd": "85",
  "numero": "999999999"
}
```

8.2.4 Excluir Funcionário

Método DELETE

Rota /funcionarios/{cpf}

Finalidade: exclui o funcionário identificado pelo CPF.

8.2.5 Enviar Código

Método POST

Rota /funcionarios/enviar-codigo

Finalidade: envia um código de verificação para o e-mail informado.

Corpo da Requisição:

```
{
  "email": "usuario@email.com",
  "codigo": "string"
}
```

8.2.6 Verificar Código

Método POST

Rota /funcionarios/verificar-codigo

Finalidade: verifica se o código informado é válido para o e-mail enviado.

Corpo da Requisição:

```
{
  "email": "usuario@email.com",
  "codigo": "string"
}
```

```
}
```

8.2.7 Reenviar Confirmação

Método POST

Rota /funcionarios/reenviar-confirmacao

Finalidade: reenvia o código de confirmação para o e-mail informado.

Corpo da Requisição:

```
{
  "email": "user@example.com",
  "codigo": "string"
}
```

8.2.8 Confirmar E-mail

Método PUT

Rota /funcionarios/confirmar-email

Finalidade: confirma o e-mail do funcionário a partir do código de verificação.

Corpo da Requisição:

```
{
  "email": "user@example.com",
  "codigo": "string"
}
```

8.2.9 Redefinir Senha

Método PATCH

Rota /funcionarios/redefinir-senha

Finalidade: redefine a senha do funcionário a partir do e-mail e da nova senha informada.

Corpo da Requisição:

```
{
  "email": "usuario@email.com",
  "novaSenha": "NovaSenha@123"
}
```

8.3 Equipamentos

8.3.1 Listar Equipamentos

Método GET
Rota /equipamentos

Finalidade: retorna a lista de equipamentos cadastrados.

8.3.2 Cadastrar Equipamento

Método POST
Rota /equipamentos

Finalidade: cadastra um novo equipamento no sistema.

Corpo da Requisição:

```
{
  "prefixo": "string",
  "descricao": "string",
  "tipo": "string",
  "cpfCadastro": "string"
}
```

8.3.3 Atualizar Equipamento

Método PUT
Rota /equipamentos/{prefixo}/{numero}

Finalidade: atualiza descrição e tipo de um equipamento identificado por prefixo e número.

Parâmetros:

Nome	Local	Descrição
prefixo	Path	Prefixo do equipamento.
numero	Path	Número do equipamento.

Corpo da Requisição:

```
{
  "descricao": "string",
  "tipo": "string"
}
```

8.3.4 Excluir Equipamento

Método DELETE

Rota /equipamentos/{prefixo}/{numero}

Finalidade: exclui um equipamento identificado por prefixo e número.

8.4 Reservas

8.4.1 Listar Reservas

Método GET

Rota /reservas

Finalidade: retorna a lista de reservas cadastradas no sistema.

8.4.2 Criar Reserva

Método POST

Rota /reservas

Finalidade: cria uma reserva para um ou mais equipamentos, vinculando-a a um funcionário responsável.

Corpo da Requisição:

```
{
  "nome": "string",
  "localidade": "string",
  "dia": "2026-07-12",
  "horarioInicio": "08:00",
  "horarioFim": "10:00",
  "cpfFuncionario": "string",
  "equipamentos": [
    {
      "prefixo": "string",
      "numero": 1
    }
  ]
}
```

8.4.3 Cancelar Reserva

Método PATCH

Rota /reservas/{id}/cancelar?cpf={cpf}

Finalidade: cancela uma reserva identificada pelo ID. A operação exige o CPF do funcionário como parâmetro de consulta.

Parâmetros:

Nome	Local	Descrição
id	Path	Identificador da reserva.
cpf	Query	CPF do funcionário responsável pela operação.

9 Execução do Backend

Esta seção descreve o procedimento necessário para executar o backend do Elook em ambiente local. O projeto utiliza Docker para simplificar a configuração do banco de dados PostgreSQL e a inicialização da API Spring Boot.

9.1 Pré-requisitos

Antes de executar o backend, é necessário possuir:

- Docker Desktop instalado;
- Docker Desktop em execução.

9.2 Configuração do Envio de E-mails

O sistema utiliza uma conta Gmail para envio de códigos de confirmação de e-mail e códigos de redefinição de senha. O projeto já possui uma conta genérica configurada para essa finalidade, mas ela pode ser alterada no arquivo:

```
backend/src/main/resources/application.properties
```

Os campos que devem ser alterados são:

```
spring.mail.username=SEU_EMAIL@gmail.com  
spring.mail.password=SUA_SENHA_DE_APLICATIVO
```

A senha utilizada deve ser uma senha de aplicativo gerada pela conta Google, e não a senha comum da conta Gmail.

9.3 Inicialização da API

Para iniciar o backend, acesse a pasta backend e execute o comando:

```
docker compose up --build
```

Na primeira execução, o Docker irá baixar as imagens necessárias, criar o banco de dados PostgreSQL, compilar o projeto Spring Boot e iniciar a API. Esse processo pode levar alguns minutos.

9.4 Acesso à Documentação Swagger

Após a inicialização da aplicação, a documentação da API poderá ser acessada em:

```
http://localhost:8080/swagger-ui/index.html
```

Essa interface só estará disponível enquanto a API estiver em execução.

9.5 Acesso ao Banco de Dados

Caso seja necessário executar comandos SQL diretamente no PostgreSQL, utilize o comando:

```
docker exec -it postgres_sistema psql -U admin_sistema -d Edook
```

Para sair do terminal do PostgreSQL, execute:

```
\q
```

9.6 Encerramento da Aplicação

Para parar os containers mantendo os dados do banco, execute:

```
docker compose down
```

Para parar os containers e remover também os dados persistidos do banco, execute:

```
docker compose down -v
```

10 Validações dos DTOs

- **E-mail:** deve possuir formato válido nos DTOs que recebem e-mail;
- **DDD:** deve conter exatamente dois dígitos;
- **Número de telefone:** deve conter oito ou nove dígitos;

- **Senha:** deve possuir no mínimo seis caracteres, incluindo letra minúscula, letra maiúscula, número e caractere especial;
- **Reserva:** exige nome, localidade, dia, horário inicial, horário final, CPF do funcionário e ao menos um equipamento;
- **Equipamento em reserva:** exige prefixo e número do equipamento;
- **Código de verificação:** exige e-mail e código preenchidos.